

State Management

In this part, we'll learn about **shared state**, which is state that is used by several different components. We will explore three approaches to managing shared state, discussing the pros and cons of each approach.

To do this, we will build a simple app containing a header that displays the user's name, with the main content also referencing the user's name. The user's name will be stored in state that needs to be accessed by several components.

We will start with the simplest state solution. This is to use one of React's state hooks to store the state and pass it to other components using props. This approach is often referred to as **prop drilling**.

The second approach we will learn about is a feature in React called **context**. We will learn how to create a context containing a state and let other components access it.

The last approach we will cover is a popular library called **Redux**. We will take the time to understand what Redux is and its concepts before refactoring the app to use it.

So, we'll cover the following topics:

- Creating the project
- Using prop drilling
- Using React context
- Using Redux

Technical requirements

We will use the following technologies in this part:

- **Node.js** and **npm**: You can install them from <https://nodejs.org/en/download/>
- **Visual Studio Code**: You can install it from <https://code.visualstudio.com/>

Creating the project

We will develop our form using Visual Studio Code and a new Create React App-based project setup. We've previously covered this several times, so we will not cover the steps in this part .

We will style the form with Tailwind CSS. We also previously covered how to install and configure Tailwind in Create React App in . So, after

you have created the React and TypeScript project, install and configure Tailwind.

The app we will build will contain a header and some content beneath it. Here is the component structure we will create:

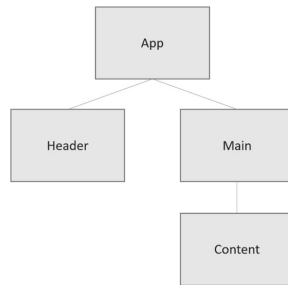


Figure 8.1 – App component structure

The header will have a **Sign in** button to authenticate and authorize a user to get their name and permissions. Once authenticated, the user's name will be displayed in the app header, and the user will be welcomed in the content. If the user has admin permissions, important content will be shown.

1. We will start by creating a file containing a function to authenticate a user. Create a folder called **api** in the **src** folder. Then, create a file called **authenticate.ts** in the **api** folder with the following content:

```
export type User = {

  id: string;

  name: string;

};

export function authenticate(): Promise<User | undefined> {

  return new Promise((resolve) =>

    setTimeout(() => resolve({ id: "1", name: "Bob" })),      1000)
```

```
);
```

```
}
```

The function simulates successful authentication for a user called Bob.

2. Next, we will create a file containing a function to authorize a user. So, create a file called **authorize.ts** in the **api** folder with the following content:

```
export function authorize(id: string): Promise<string[]> {
```

```
  return new Promise((resolve) =>
```

```
    setTimeout(() => resolve(["admin"]), 1000)
```

```
  );
```

```
}
```

The function simulates a user being authorized to have admin permissions.

3. Next, we will create a component for the app header. Create a file called **Header.tsx** in the **src** folder with the following content:

```
import { User } from '../api/authenticate';
```

```
type Props = {
```

```
  user: undefined | User;
```

```
  onSignInClick: () => void;
```

```
loading: boolean;
```

```
};
```

The component has a prop for the user, which will be **undefined** if the user isn't authenticated yet. The component also has a prop called **onSignInClick** for a **Sign in** button click handler. The last component prop is called **loading** and it determines whether the app is in a loading state when the user is authenticated or authorized.

4. Add the following component implementation into **Header.tsx**:

```
export function Header({
```

```
  user,
```

```
  onSignInClick,
```

```
  loading,
```

```
}: Props) {
```

```
  return (
```

```
    <header className="flex justify-between items-center border-b-2 border-gray-100 py-
```

```
      {user ? (
```

```
        <span className="ml-auto font-bold">
```

```
          {user.name} has signed in
```

```

    </span>

    ) : (

    <button

        onClick={onSignInClick}

        className="whitespace-nowrap inline-flex items-center justify-center ml-2"

        disabled={loading}

    >

        {loading ? '...' : 'Sign in'}

    </button>

    )}

</header>

);

}

```

The component informs the user that they have signed in if they have been authenticated. If the user is unauthenticated, the component displays a **Sign in** button.

5. Next, we will implement a component for the main app content.
Create a file called **Main.tsx** in the **src** folder with the following content:

```
import { User } from './api/authenticate';

import { Content } from './Content';

type Props = {

  user: undefined | User;

  permissions: undefined | string[];

};
```

The component has a prop for the user and their permissions. We have imported a component called **Content**, which we will create in *step 7*.

6. Now, add the following component implementation in **Main.tsx**:

```
export function Main({ user, permissions }: Props) {

  return (

    <main className="py-8">

      <h1 className="text-3xl text-center font-bold underline">Welcome</h1>

      <p className="mt-8 text-xl text-center">

        {user ? `Hello ${user.name}!` : "Please sign in"}
```

```
</p>
```

```
<Content permissions={permissions} />
```

```
</main>
```

```
);
```

```
}
```

The component instructs the user to sign in if they are unauthenticated or shows a **Hello** message if they are authenticated. The content also references a **Content** component passing it the user's permissions.

7. The last file to create in the **src** folder is called **Content.tsx**. Add the following content to the file:

```
type Props = {
```

```
  permissions: undefined | string[];
```

```
};
```

```
export function Content({ permissions }: Props) {
```

```
  if (permissions === undefined) {
```

```
    return null;
```

```
  }
```

```
return permissions.includes('admin') ? (
```

```
<p className="mt-4 text-l text-center">
```

```
    Some important stuff that only an admin can do
```

```
</p>
```

```
) : (
```

```
<p className="mt-4 text-l text-center">
```

```
    Insufficient permissions
```

```
</p>
```

```
);
```

```
}
```

If the user is unauthorized, the component displays nothing. If the user has admin permissions, it displays some important stuff. Otherwise, it informs the user that they lack permissions.

That completes the project setup. The app will compile and run but won't show any of the components we created yet because we haven't referenced them in the **App** component. We will do this next when we share the user and permission information across several components.

Using prop drilling

In this first state management approach, we will store the **user**, **permissions**, and **loading** state in the **App** component. The **App** component will then pass this state to the **Header** and **Main** components using props.

So, this approach uses React features that we are already aware of. The approach is referred to as **prop drilling** because the state is passed down the component tree using props.

Carry out the following steps to rework the **App** component to store the **user**, **permissions**, and **loading** state, and pass this state down to the **Header** and **Main** components:

1. Open **App.tsx** and start by removing all the existing code and adding the following import statements:

```
import { useReducer } from 'react';

import { Header } from './Header';

import { Main } from './Main';

import { authenticate, User } from './api/authenticate';

import { authorize } from './api/authorize';
```

We have imported **useReducer** from React to store the state. We have also imported the **Header** and **Main** components so that we can render them with the state values. Lastly, we've imported the **authenticate** and **authorize** functions because we will create the **Sign in** handler in this component.

2. After the import statements, add a type for the state and create a variable for the initial state values:

```
type State = {

  user: undefined | User,

  permissions: undefined | string[],

  loading: boolean,
```

```
};
```

```
const initialState: State = {
```

```
  user: undefined,
```

```
  permissions: undefined,
```

```
  loading: false,
```

```
};
```

3. Next, create a type for the different actions that can update the state:

```
type Action =
```

```
  | {
```

```
    type: "authenticate",
```

```
  }
```

```
  | {
```

```
    type: "authenticated",
```

```
    user: User | undefined,
```

```
  }
```

```

| {

    type: "authorize",

}

| {

    type: "authorized",

    permissions: string[],

};

```

The **"authenticate"** action will start the authentication process, and **"authenticated"** happens when it has been completed. Likewise, the **"authorize"** action will start the authorization process, and **"authorized"** happens when it has been completed.

4. Next, add a **reducer** function that updates the state:

```

function reducer(state: State, action: Action): State {

    switch (action.type) {

        case "authenticate":

            return { ...state, loading: true };

        case "authenticated":

```

```

        return { ...state, loading: false, user: action.user };

    case "authorize":

        return { ...state, loading: true };

    case "authorized":

        return {

            ...state,

            loading: false,

            permissions: action.permissions,

        };

    default:

        return state;

    }

}

```

The function takes in the existing state and the action as parameters. The function uses a switch statement on the action type to create a new version of the state in each branch.

5. Now, let's define the **App** component as follows:

```
function App() {

  const [{ user, permissions, loading }, dispatch] =

    useReducer(reducer, initialState);

  return (

    <div className="max-w-7xl mx-auto px-4">

      <Header

        user={user}

        onSignInClick={handleSignInClick}

        loading={loading}

      />

      <Main user={user} permissions={permissions} />

    </div>

  );

}
```

```
export default App;
```

The component uses **useReducer** with the **reducer** function and the **initialState** variable we defined earlier. We have destructured the **user**, **permissions**, and **loading** state values from **useReducer**. In the JSX, we have rendered both the **Header** and **Main** components passing the appropriate state values as props.

6. The **Header** element in the JSX references a handler called **handleSignInClick**, which needs implementation. Create this above the return statement as follows:

```
async function handleSignInClick() {

    dispatch({ type: "authenticate" });

    const authenticatedUser = await authenticate();

    dispatch({

        type: "authenticated",

        user: authenticatedUser,

    });

    if (authenticatedUser !== undefined) {

        dispatch({ type: "authorize" });

        const authorizedPermissions = await authorize(

            authenticatedUser.id
```

```

    );

    dispatch({

      type: "authorized",

      permissions: authorizedPermissions,

    });

  }

}

```

The sign-in handler authenticates and authorizes the user and dispatches the necessary actions along the way.

7. Run the app in development mode by running **npm start** in the terminal. The app appears as shown in the screenshot:

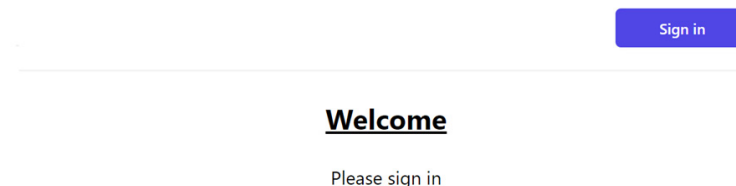


Figure 8.2 – App before signing in

8. Click the **Sign in** button. The authentication and authorization processes then happen, and after a couple of seconds, the following screen appears:

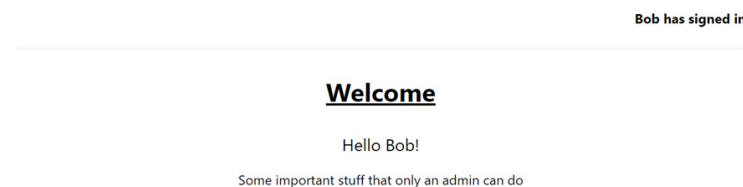


Figure 8.3 – App after signing in

That completes the prop drilling approach.

A nice thing about this approach is that it is simple and uses React features we are already familiar with. A downside of this approach is that it forces all components between the component providing state and components accessing the state to have a prop for that state. So, some components that do not need access to the state are forced to access it. An example is the **Main** component – the **permissions** state is forced to pass through it to the **Content** component.

The key point in this section is that it is fine to share state across a few adjacent components using props but isn't ideal for sharing across lots of components far apart in the component tree.

Next, keep the app running, and we will look at a more appropriate solution for sharing state across many components.

Using React context

In this section, we will learn a feature in React called **context**. We will then refactor the app from the last section to use React context.

Understanding React context

React context is an object that can be accessed by components. This object can contain state values, so it provides a mechanism for sharing state across components.

A context is created using a **createContext** function as follows:

```
const SomeContext = createContext<ContextType>(defaultValue);
```

A default value for the context must be passed into **createContext**. It also has a generic type parameter for the type that represents the object created by **createContext**.

The context also contains a **Provider** component that needs to be placed above components requiring access to the context object in the component tree. A provider wrapper component can be created that stores the shared state and passes it to the context **Provider** component as follows:

```
export function SomeProvider({ children }: Props) {
  const [someState, setSomeState] = useState(initialState);
  return (
    <SomeContext.Provider value={{ someState }}>
      {children}
    </SomeContext.Provider>
  );
}
```

useState has been used for the state in the preceding example, but **useReducer** could also be used.

The provider wrapper component can then be placed appropriately in the component tree, above components requiring the shared state:

```
function App() {
  return (
    <SomeProvider>
      <Header />
      <Main />
    </SomeProvider>
  );
}
```

React also contains a **useContext** hook that can be used so that the context values can be consumed as a hook, as follows:

```
const { someState } = useContext(SomeContext);
```

The context must be passed into **useContext** and properties from the context object can be destructured from its result.

So, components that want access to the shared state can access it using **useContext** as follows:

```
export function SomeComponent() {
  const { someState } = useContext(SomeContext);
  return <div>I have access to {someState}</div>;
}
```

For more information on React context, see the following link:

<https://reactjs.org/docs/context.html>.

Now that we understand React context, we will use it in the app we created in the previous section.

Using React context

We will refactor the app from the last section to use React context. We will start by creating a file containing the context and the provider wrapper. Then, we will use **useReducer** in the provider wrapper to store the state. We will also create a wrapper for **useContext** to make consuming it easy.

So, to do this, carry out the following steps:

1. Start by creating a file called **AppContext.tsx** in the **src** folder. This will contain the context, the provider wrapper, and the **useContext** wrapper.
2. Add the following import statements to **AppContext.tsx**:

```
import {  
  
  createContext,  
  
  useContext,  
  
  useReducer,  
  
  ReactNode,  
  
} from 'react';  
  
import { User } from '../api/authenticate';
```

We have imported all the functions we need from React along with the **ReactNode** type that we will need for the provider wrapper **children** prop. We have also imported the **User** type, which we will need for the user state type.

3. We need to add a type for the state and a variable for the initial state values. We already have these in **App.tsx**, so the following lines can be moved from **App.tsx** to **AppContext.tsx**:

```
type State = {  
  
  user: undefined | User,  
  
  permissions: undefined | string[],  
  
  loading: boolean,  
  
};
```

```

const initialState = {

  user: undefined,

  permissions: undefined,

  loading: false,

};

```

4. Similarly, the **Action** type and the **reducer** function can be moved from **App.tsx** to **AppContext.tsx**. Here are the lines to move:

```

type Action =

  | {

    type: "authenticate",

  }

  | {

    type: "authenticated",

    user: User | undefined,

  }

  | {

```

```
    type: "authorize",
```

```
  }
```

```
| {
```

```
    type: "authorized",
```

```
    permissions: string[],
```

```
};
```

```
function reducer(state: State, action: Action): State {
```

```
  switch (action.type) {
```

```
    case "authenticate":
```

```
      return { ...state, loading: true };
```

```
    case "authenticated":
```

```
      return { ...state, loading: false, user: action.user };
```

```
    case "authorize":
```

```
      return { ...state, loading: true };
```

```

        case "authorized":

            return { ...state, loading: false, permissions: action.permissions };

        default:

            return state;

    }

}

```

Note that the **App.tsx** file will raise a compile error after moving this function. We will resolve this in the next set of instructions.

5. Next, we will create a type for the context in **AppContext.tsx**:

```

type AppContextType = State & {

    dispatch: React.Dispatch<Action>,

};

```

The context will consist of the state values and a **dispatch** function to dispatch actions.

6. Now we can create the context as follows:

```

const AppContext = createContext<AppContextType>({

    ...initialState,

    dispatch: () => {},

```

```
});
```

We have called the context **AppContext**. We use the **initialState** variable and a dummy **dispatch** function as the default context value.

7. Next, we can implement the provider wrapper as follows:

```
type Props = {

  children: ReactNode;

};

export function AppProvider({ children }: Props) {

  const [{ user, permissions, loading }, dispatch] =

    useReducer(reducer, initialState);

  return (

    <AppContext.Provider

      value={{

        user,

        permissions,

        loading,
```

```

        dispatch,

      }}

    >

    {children}

  </AppContext.Provider>

);

}

```

We have called the component **AppProvider**, and it returns the context's **Provider** component with the state values and the **dispatch** function from **useReducer**.

8. The last thing to do in **AppContext.tsx** is to create a wrapper for **useContext** as follows:

```
export const useAppContext = () => useContext(AppContext);
```

That completes the work we need to do in **AppContext.tsx**.

So, **AppContext.tsx** exports an **AppProvider** component that can be placed above **Header** and **Main** in the component tree so that they can access the user and permissions information. **AppContext.tsx** also exports **useAppContext** so that the **Header**, **Main**, and **Content** components can use it to get access to the user and permissions information.

Now, carry out the following steps to make the necessary changes to the **App**, **Header**, **Main**, and **Content** components to access the user and permissions information from **AppContext**:

1. We will start with **Header.tsx**. Begin by importing the **authenticate**, **authorize**, and **useAppContext** functions. Also, remove the **User** type and the props for the **Header** component:

```
import { authenticate } from './api/authenticate';
```

```
import { authorize } from './api/authorize';
```

```
import { useAppContext } from './AppContext';
```

```
export function Header() {
```

```
  return ...
```

```
}
```

2. **Header** will now handle the sign-in process instead of **App**. So, move the **handleSignInClick** handler from **App** in **App.tsx** to **Header.tsx** and place it above the return statement as follows:

```
export function Header() {
```

```
  async function handleSignInClick() {
```

```
    dispatch({ type: 'authenticate' });
```

```
    const authenticatedUser = await authenticate();
```

```
    dispatch({
```

```
      type: 'authenticated',
```

```
      user: authenticatedUser,
```

```
    });
```

```

    if (authenticatedUser !== undefined) {

        dispatch({ type: 'authorize' });

        const authorizedPermissions = await authorize(

            authenticatedUser.id

        );

        dispatch({

            type: 'authorized',

            permissions: authorizedPermissions,

        });

    }

}

return ...

}

```

3. Update the sign-in click handler to reference the function we just added:

```

<button

```

```
onClick={handleSignInClick}
```

```
className=...
```

```
disabled={loading}
```

```
>
```

```
{loading ? '...' : 'Sign in'}
```

```
</button>
```

4. The last thing to do in **Header.tsx** is to get **user**, **loading**, and **dispatch** from the context. Add the following call to **useAppContext** at the top of the component:

```
export function Header() {
```

```
  const { user, loading, dispatch } = useAppContext();
```

```
  ...
```

```
}
```

5. Let's move on to **Main.tsx**. Remove the import statement for the **User** type and add an import statement for **useAppContext**:

```
import { Content } from './Content';
```

```
import { useAppContext } from './AppContext';
```

6. Remove the props for the **Main** component and get **user** from **useAppContext**:

```

export function Main() {

    const { user } = useAppContext();

    return ...

}

```

7. In the JSX in **Main**, remove the **permissions** attribute on the **Content** element:

```

<Content />

```

8. Now, open **Content.tsx** and add an import statement for **useAppContext**:

```

import { useAppContext } from './AppContext';

```

9. Remove the props for the **Content** component and get **permissions** from **useAppContext**:

```

export function Content() {

    const { permissions } = useAppContext();

    if (permissions === undefined) {

        return null;

    }

    return ...

}

```

10. Lastly, we will modify **App.tsx**. Remove the import statements except for **Header** and **Main**, and add an import statement for **AppProvider**:

```
import { Header } from './Header';
```

```
import { Main } from './Main';
```

```
import { AppProvider } from './AppContext';
```

11. Still in **App.tsx**, remove the call to **useReducer** and remove all the attributes passed to **Header** and **Main**:

```
function App() {  
  
  return (  
  
    <div className="max-w-7xl mx-auto px-4">  
  
      <Header />  
  
      <Main />  
  
    </div>  
  
  );  
  
}
```

12. Wrap **AppProvider** around **Header** and **Main** so that they can access the context:

```
function App() {
```

```

    return (

      <div className="max-w-7xl mx-auto px-4">

        <AppProvider>

          <Header />

          <Main />

        </AppProvider>

      </div>

    );

  }

```

The compile errors will now be resolved and the running app will look and behave like before.

13. Stop the app running by pressing *Ctrl + C*.

That completes the refactoring of the app to use React context instead of prop drilling.

In comparison to prop drilling, React context requires more code to be written. However, it allows components to access shared state using a hook rather than passing it through components using props. It's an elegant, shared-state solution, particularly when many components share state.

Next, we will learn about a popular third-party library that can be used to share state.

Using Redux

In this section, we will learn about Redux before using it to refactor the app we have been working on to use it.

Understanding Redux

Redux is a mature state management library that was first released in 2015. It was released before React context and became a popular approach for shared state management.

Creating a store

In Redux, the state lives in a centralized immutable object referred to as a **store**. There is only a single store for the whole app. Like **useReducer**, the state in a store is updated by dispatching an **action**, which is an object containing the type of change and any data required to make the change. An action is handled by a **reducer** function, which creates a new version of the state.

In the past, a lot of code was needed to set up a Redux store and consume it in a React component. Today, a companion library called Redux Toolkit reduces the code required to use Redux. A Redux store can be created using the Redux Toolkit's **configureStore** function as follows:

```
export const store = configureStore({
  reducer: {
    someFeature: someFeatureReducer,
    anotherFeature: anotherFeatureReducer
  },
});
```

The **configureStore** function takes in the store's reducers. Each feature in the app can have its own area of state and reducer to change the state. The different areas of state are often referred to as **slices**. In the preceding example, there are two slices called **someFeature** and **anotherFeature**.

The Redux Toolkit has a function to create slices, called **createSlice**:

```
export const someSlice = createSlice({
  name: "someFeature",
  initialState,
  reducers: {
    someAction: (state) => {
      state.someValue = "something";
    },
    anotherAction: (state) => {
      state.someOtherValue = "something else";
    },
  },
});
```

The **createSlice** function takes in an object parameter containing the slice name, the initial state, and functions to handle the different actions and update the state.

The slice created from **createSlice** contains a **reducer** function that wraps the action handlers. This **reducer** function can be referenced in the **reducer** property of **configureStore** when the store is created:

```
export const store = configureStore({
  reducer: {
    someFeature: someSlice.reducer,
    ...
  },
});
```

In the preceding code snippet, the reducer from **someSlice** has been added to the store.

Providing the store to React components

The Redux store is defined in the component tree using its **Provider** component. The value of the Redux store (from **configureStore**) needs to be specified on the **Provider** component. The **Provider** component must be placed above the components requiring access to the store:

```
<Provider store={store}>
  <SomeComponent />
  <AnotherComponent />
</Provider>
```

In the preceding example, **SomeComponent** and **AnotherComponent** have access to the store.

Accessing the store from a component

Components can access state from the Redux store using a **useSelector** hook from React Redux. A function that selects the relevant state in the store is passed into **useSelector**:

```
const someValue = useSelector(
  (state: RootState) => state.someFeature.someValue
);
```

In the preceding example, **someValue** is selected from the **someFeature** slice in the store.

Dispatching actions to the store from a component

React Redux also has a **useDispatch** hook that returns a **dispatch** function that can be used to dispatch actions. The action is a function from the slice created using **createSlice**:

```
const dispatch = useDispatch();
return (
  <button onClick={() => dispatch(someSlice.actions.someAction())}>
    Some button
  </button>
);
```

In the preceding example, **someAction** in **someSlice** is dispatched when the button is clicked.

For more information on Redux, see the following link: <https://redux.js.org/>. And for more information on the Redux Toolkit, see the following link: <https://redux-toolkit.js.org/>.

Now that we understand Redux, we will use it in the app we created in the previous section.

Installing Redux

First, we must install Redux and the Redux Toolkit into our project. Run the following command in the terminal:

```
npm i @reduxjs/toolkit react-redux
```

This will install all the Redux bits we need, including its TypeScript types.

Using Redux

Now, we can refactor the app to use Redux instead of React context. First, we will create a Redux slice for the user information before creating a Redux store with this slice. We will then move on to add the store to the React component tree and consume it in the **Header**, **Main**, and **Content** components.

Creating a Redux Slice

We will start by creating a Redux slice for the state for a user. Carry out the following steps:

1. Create a folder called **store** in the **src** folder and then a file called **userSlice.ts** within it.
2. Add the following import statements to **userSlice.ts**:

```
import { createSlice } from '@reduxjs/toolkit';
```

```
import type { PayloadAction } from '@reduxjs/toolkit';
```

```
import { User } from '../api/authenticate';
```

We will eventually use **createSlice** to create the Redux slice.

PayloadAction is a type that we can use for action objects. We will need the **User** type when defining the type for the state.

3. Copy the following **State** type and initial state value from **AppContext.tsx** into **userSlice.ts**:

```
type State = {

  user: undefined | User;

  permissions: undefined | string[];

  loading: boolean;

};

const initialState: State = {

  user: undefined,

  permissions: undefined,

  loading: false,

};
```

4. Next, start to create the slice in **userSlice.ts**, as follows:

```

export const userSlice = createSlice({

  name: 'user',

  initialState,

  reducers: {

  }

});

```

We have named the slice **user** and passed in the initial state value. We export the slice so that we can use it later to create the Redux store.

5. Now, define the following action handlers inside the **reducers** object:

```

reducers: {

  authenticateAction: (state) => {

    state.loading = true;

  },

  authenticatedAction: (

    state,

    action: PayloadAction<User | undefined>

```

```
) => {

    state.user = action.payload;

    state.loading = false;

},

authorizeAction: (state) => {

    state.loading = true;

},

authorizedAction: (

    state,

    action: PayloadAction<string[]>

) => {

    state.permissions = action.payload;

    state.loading = false;

}
```

```
}
```

Each action handler updates the required state. **PayloadAction** is used for the type of the action parameters. **PayloadAction** is a generic type with a parameter for the type of the action payload.

6. Lastly, export the action handlers and the **reducer** function from the slice:

```
export const {  
  
  authenticateAction,  
  
  authenticatedAction,  
  
  authorizeAction,  
  
  authorizedAction,  
  
} = userSlice.actions;  
  
export default userSlice.reducer;
```

A default export has been used for the **reducer** function so the consumer can name it as required.

That completes the implementation of the Redux slice.

Creating the Redux store

Next, let's create the Redux store. Carry out the following steps:

1. Create a file called **store.ts** in the **store** folder containing the following import statements:

```
import { configureStore } from '@reduxjs/toolkit';  
  
  
  
import userReducer from '../userSlice';
```

2. Next, use the **configureStore** function to create the store referencing the reducer from the slice we created earlier:

```
export const store = configureStore({

  reducer: { user: userReducer }

});
```

We export the **store** variable so that we can later use it on React Redux's **Provider** component.

3. The last thing to do in **store.ts** is export the type for Redux's full state object, which we will eventually require in the **useSelector** hook in components consuming the Redux store:

```
export type RootState = ReturnType<typeof store.getState>;
```

ReturnType is a standard TypeScript utility type that returns the return type of the function type passed into it. The **getState** function in the Redux store returns the full state object. So, we use **ReturnType** to infer the type of the full state object rather than explicitly defining it.

That completes the implementation of the Redux store.

Adding the Redux store to the component tree

Next, we will add the store at an appropriate place in the component tree using the **Provider** component from React Redux. Follow these steps:

1. Open **App.tsx** and remove the **AppContext** import statement. Remove the **AppContext.tsx** file as well because this is no longer required.
2. Add an import statement for the **Provider** component from React Redux and the Redux store we created:

```
import { Provider } from 'react-redux';

import { store } from '../store/store';
```

3. Replace **AppProvider** with **Provider** in the JSX, as follows:

```
<div className="max-w-7xl mx-auto px-4">
```

```
<Provider store={store}>
```

```
<Header />
```

```
<Main />
```

```
</Provider>
```

```
</div>
```

We pass the imported Redux store into **Provider**.

The Redux store is now accessible to the **Header**, **Main**, and **Content** components.

Consuming the Redux store in the components

We will now integrate the Redux store into the **Header**, **Main**, and **Content** components. This will replace the previous React context consumption code. Follow these steps:

1. Start by opening **Header.tsx** and remove the **AppContext** import statement.
2. Add the following import statements to **Header.tsx**:

```
import { useSelector, useDispatch } from 'react-redux';
```

```
import type { RootState } from './store/store';
```

```
import {
```

```
  authenticateAction,
```

```
  authenticatedAction,
```

```
authorizeAction,
```

```
authorizedAction,
```

```
} from './store/userSlice';
```

We will be referencing state from Redux as well as dispatching actions, so we have imported both **useSelector** and **useDispatch**. The **RootState** type is required in the function we will eventually pass to **useSelector**. We have also imported all the actions from the slice we created because we will need them in the revised sign-in handler.

3. Inside the **Header** component, replace the **useAppContext** call with **useSelector** calls to get the required state:

```
export function Header() {
```

```
  const user = useSelector(  
  

```

```
    (state: RootState) => state.user.user  
  

```

```
  );
```

```
  const loading = useSelector(  
  

```

```
    (state: RootState) => state.user.loading  
  

```

```
  );
```

```
  async function handleSignInClick() {  
  

```

```
    ...  
  }  
}
```

```
}
```

```
return ...
```

```
}
```

4. Also, call **useDispatch** to get a **dispatch** function:

```
export function Header() {
```

```
  const user = useSelector(
```

```
    (state: RootState) => state.user.user
```

```
  );
```

```
  const loading = useSelector(
```

```
    (state: RootState) => state.user.loading
```

```
  );
```

```
  const dispatch = useDispatch();
```

```
  async function handleSignInClick() {
```

```
    ...
```

```
}
```

```
return ...
```

```
}
```

5. The last thing to do in **Header.tsx** is to modify **handleSignInClick** to reference the action functions from the Redux slice:

```
async function handleSignInClick() {
```

```
  dispatch(authenticateAction());
```

```
  const authenticatedUser = await authenticate();
```

```
  dispatch(authenticatedAction(authenticatedUser));
```

```
  if (authenticatedUser !== undefined) {
```

```
    dispatch(authorizeAction());
```

```
    const authorizedPermissions = await authorize(
```

```
      authenticatedUser.id
```

```
    );
```

```
    dispatch(authorizedAction(authorizedPermissions));
```

```
  }
```

```
}
```

6. Now, open **Main.tsx** and replace the **AppContext** import statement with import statements for **useSelector** and the **RootState** type:

```
import { useSelector } from 'react-redux';
```

```
import { RootState } from '../store/store';
```

7. Replace the call to **useAppContext** with a call to **useSelector** to get the **user** state value:

```
export function Main() {
```

```
  const user = useSelector(  
    (state: RootState) => state.user.user
```

```
  );
```

```
  return ...
```

```
}
```

8. Next, open **Content.tsx** and replace the **AppContext** import statement with import statements for **useSelector** and the **RootState** type:

```
import { useSelector } from 'react-redux';
```

```
import { RootState } from '../store/store';
```

9. Replace the call to **useAppContext** with a call to **useSelector** to get the **permissions** state value:

```
export function Content() {
```

```
  const permissions = useSelector(  
    (state: RootState) => state.permissions
```

```

    (state: RootState) => state.user.permissions

    );

    if (permissions === undefined) {

        return null;

    }

    return ...

}

```

10. Run the app by running **npm start** in the terminal. The app will look and behave just as it did before.

That completes the refactoring of the app to use Redux rather than React context.

Here's a recap of the key points for using Redux:

- State is stored in a central store
- State is updated by dispatching actions that are handled by reducers
- A **Provider** component needs to be placed appropriately in the component tree to give components access to the Redux store
- Components can select state using a **useSelector** hook and dispatch actions using a **useDispatch** hook

As you have experienced, even using the Redux Toolkit requires many steps when using Redux to manage state. It is overkill for simple state management requirements but shines when there is a lot of shared application-level state.

Summary

In this part, we built a small one-page app that contained components that needed to share state. We started by using our existing knowledge and used props to pass the state between the components. We learned that a problem with this approach was that components not needing ac-

cess to the state are forced to access it if its child components do need access to it.

We moved on to learn about React context and refactored the app to use it. We learned that React context can store state using **useState** or **useReducer**. The state can then be provided to components in the tree using the context's **Provider** component. Components then access the context state via the **useContext** hook. We found that this was a much nicer solution than passing the state via props, particularly when many components need access to the state.

Next, we learned about Redux, which is similar to React context. A difference is that there can only be a single Redux store containing the state, but there can be many React contexts. We learned that a **Provider** component needs to be added to the component tree to give components access to the Redux store. Components select state using the **useSelector** hook and dispatch actions using the **useDispatch** hook. Reducers handle actions and then update the state accordingly.

In the next part, we will learn how to work with REST APIs in React.

Questions

Answer the following questions to check what you have learned in this part:

1. We have a context defined as follows to hold the theme state for an app:

```
type Theme = {  
  
  name: string;  
  
  color: 'dark' | 'light';  
  
};  
  
type ThemeContextType = Theme & {  
  
  changeTheme: (
```

```

    name: string,

    color: 'dark' | 'light'

  ) => void;

};

const ThemeContext = createContext<ThemeContextType>();

```

The code doesn't compile though; what is the problem?

2. The context from question 1 has a provider wrapper called **ThemeProvider**, which is added to the component tree as follows:

```

<ThemeProvider>

  <Header />

  <Main />

</ThemeProvider>

<Footer />

```

The theme state is **undefined** when destructured from **useContext** in the **Footer** component. What is the problem?

3. Is it possible to have two React contexts in an app?
4. Is it possible to have two Redux stores in an app?
5. The following code dispatches an action to change the theme:

```

function handleChangeTheme({ name, color }: Theme) {

```

```

        useDispatch(changeThemeAction(name, color));

    }

```

There is a problem with this code. What is this problem?

6. In a React component, is it possible to use state only required by this component using **useState** as well as state from a Redux store?
7. In this part, when we implemented the Redux slice, the action handlers appeared to directly update the state, as in the following example:

```

authorizedAction: (

    state,

    action: PayloadAction<string[]>

) => {

    state.permissions = action.payload;

    state.loading = false;

}

```

Why are we allowed to mutate the state? I thought that state in React had to be immutable?

Answers

1. **createContext** must be passed a default value when using it with TypeScript. Here's the corrected code:

```

const ThemeContext = createContext<ThemeContextType>({

```

```
name: 'standard',
```

```
color: 'light',
```

```
changeTheme: (name: string, color: 'dark' | 'light') => {},
```

```
});
```

2. **Footer** must be placed inside **ThemeProvider** as follows:

```
<ThemeProvider>
```

```
<Header />
```

```
<Main />
```

```
<Footer />
```

```
</ThemeProvider>
```

3. Yes, there is no limit on the number of React contexts in an app.

4. No, only a single Redux store can be added to an app.

5. **useDispatch** can't be used directly to dispatch an action – it returns a function that can be used to dispatch an action:

```
const dispatch = useDispatch();
```

```
function handleChangeTheme({ name, color }: Theme) {
```

```
  dispatch(changeThemeAction(name, color));
```

```
}
```

6. Yes, local state defined using **useState** or **useReducer** can be used alongside shared state from a Redux store.
7. The Redux Toolkit uses a library called **immer** to allow developers to directly update the **state** object without mutating it. *immer*.